

Scheduling

- Problem definition:
 - One CPU with a number of processes/task. Only one process can use the CPU at a time and each process/task is specialized in one job. What's the best way to organize the processes (schedule them) ?
 - How will the CPU time be divided among the processes/task competing to use it ?

Scheduling

- A set of policies and mechanisms built into the operating system that govern the order in which the work to be done by a computer system is completed.
- Primary objective of scheduling is to optimize system performance.

Scheduling - Goals

- **Utilization / Efficiency**
 - Keep the CPU busy 100% of the time with useful work
- **Throughput**
 - Maximize the number of jobs processed per hour
- **Turnaround time**
 - From time of submission to the time of completion, minimize the time batch users must wait for output
- **Waiting time**
 - Sum of times spent in ready queue - Minimize this
- **Response Time**
 - Time from submission till the first response is produced, minimize response time for interactive users
- **Fairness**
 - Make sure each process gets a fair share of the CPU

Schedulers

- Multiprogramming : *CPU utilization maximized.*
- Time sharing : switch CPU among process so frequently that **users can interact** with each program while it running.
- Scheduling
 - Assigning a particular process for CPU usage
- Types
 - Long term
 - Short Term
 - Medium term

Schedulers

- OS module that **selects the next job** to be admitted into system / next process to run.
- Piece of OS code that decides *priorities* of processes and **how long each will run**
- A process migrates between the various scheduling queues throughout its lifetime.
- OS selects processes from queues.
- Selection of processes - by *appropriate schedulers*.

Schedulers...

Long –term scheduler or Job scheduler : selects processes and loads them into memory for execution.

Short – term scheduler or CPU scheduler: selects from among the processes that are ready to execute and allocates the CPU to one of them.

Primary distinction between the two: Frequency of their execution.

- Short – term scheduler must select a **new process for the CPU** frequently.
- Process may execute for few milliseconds before waiting for I/O request.
- Executes **at least once for every 100 m.sec.**
- If it takes 10ms to decide to execute a process for 100ms, then $10(100+10) = 9\%$ of CPU is being used (wasted) for scheduling the work.

Schedulers...

Long –term scheduler - executes much less frequently.

- Takes time for creation of new process in the system.
- Controls the degree of multiprogramming - the no. of processes in memory.
- If degree of multiprogramming is stable, then avg. no of process creation must be equal to the avg. departure rate of processes leaving the system.
- Long – term scheduler to be invoked only when process leaves the system.
- Selects a good process mix of I/O – bound and CPU- bound processes.

Scheduling algorithms

Scheduling algorithms

Non-Preemptive:

- Once a process enters running state, it is not removed from processor until it completes its service time.
- Context switch only when process terminates or blocks.

Preemptive:

- Notion of prioritized computation.
- If a new process with higher priority enters ready list, process currently executing removed and returned to ready list.

Recap

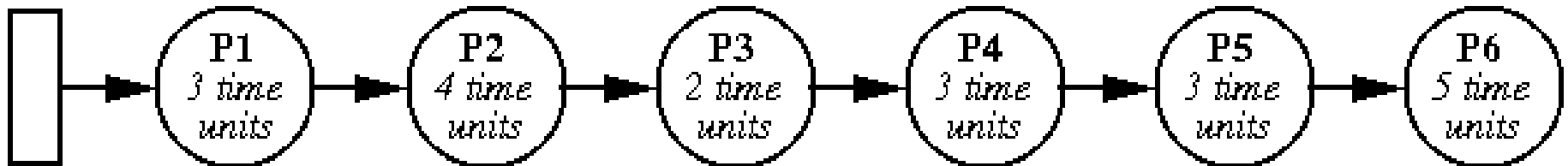
- Process states
- Scheduling - Goals
- Schedulers
 - Long Term, Short term and medium term
- Scheduling algorithm – approaches
 - Preemptive and non- preemptive

Scheduling algorithms

- FCFS
- SJF
- Round Robin
- SRTN
- Multilevel feedback queues

FCFS/ FIFO

- One ready queue
- OS runs processes at the head of the Q
- New processes come in at end of Q
- Running process continues till it finishes or performs I/O



FCFS/ FIFO

- EXAMPLE DATA:

	Process	Arrival Time	Service Time
FCFS	1	0	8
	2	1	4
	3	2	9
	4	3	5



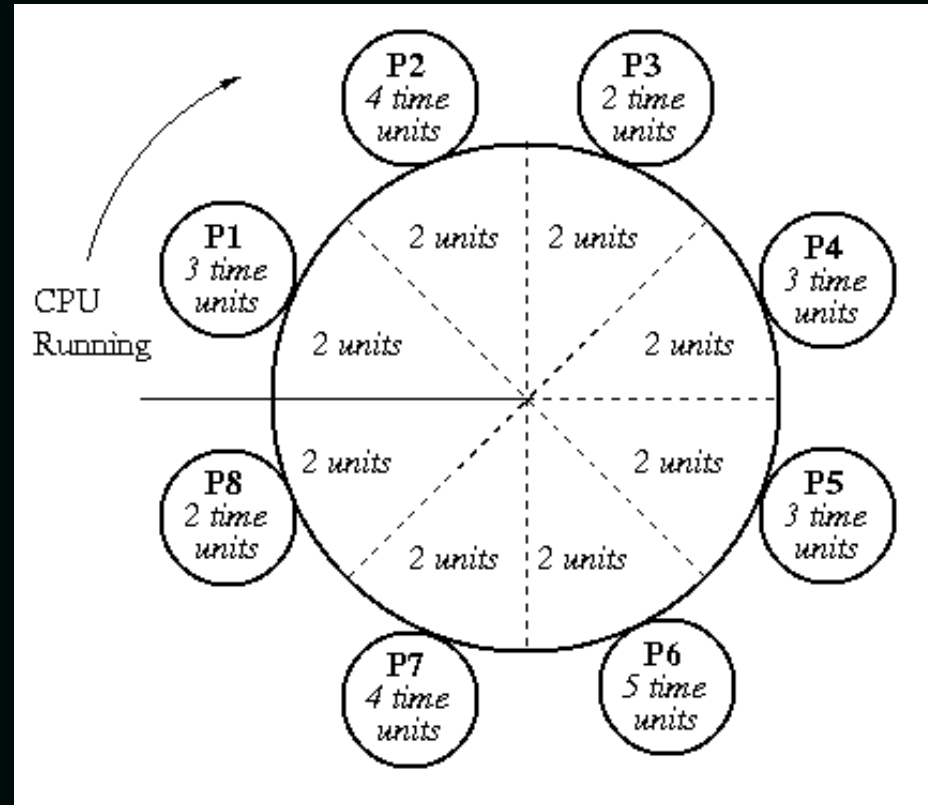
$$\text{Average turnaround time} = (8-0) + (12-1) + (21-2) + (26-3) / 4 = 61/4 = 15.25$$

Problem: One process can monopolize CPU.

Solution: Limit maximum amount of time (time slice) that a process can run without a context switch.

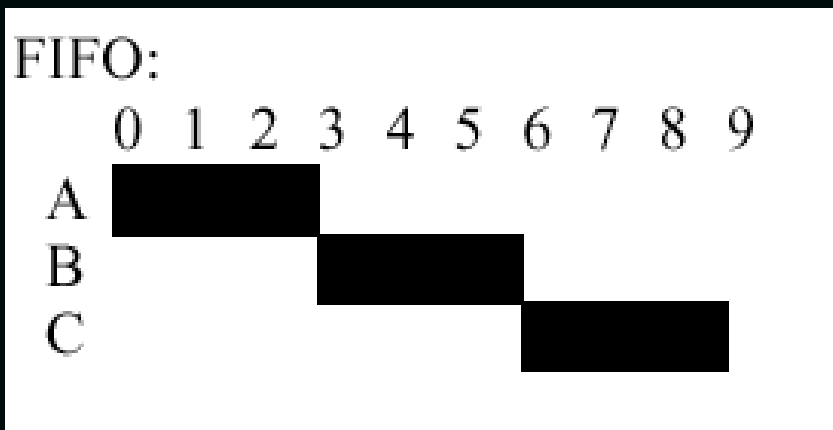
Round Robin

- Used mostly on timeshared systems
- Allows multiple users slices of the CPU on a “round robin” basis
- Majority of users have the same priority
- Rule of thumb :
Time slice > CPU burst

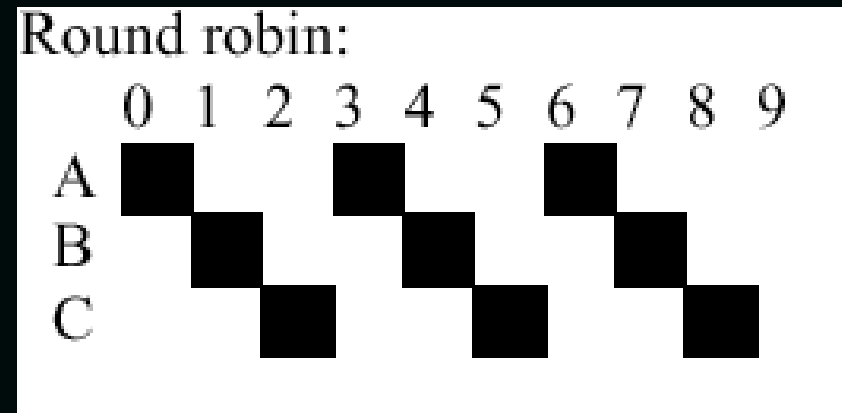


Implementation of priorities: run highest priority processes first, use round-robin among processes of equal priority. Re-insert process in run queue behind all processes of greater or equal priority

FIFO vs Round robin



Process A finishes after 3 units, B 6, and C 9. The average is $(3+6+9)/3 = 6$ units

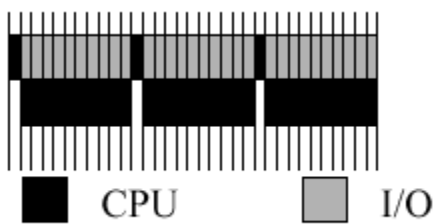


Process A finishes after 7 units, B 8, and C 9, so the average is $(7+8+9)/3 = 8$ units

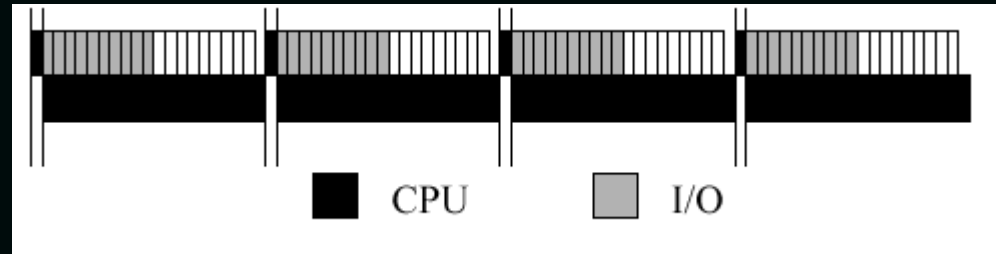
Role of timeslice

- Process 1 – I/O bound – uses 1 ms CPU then 10 ms I/O
- Process 2 – CPU bound

Two scenarios :

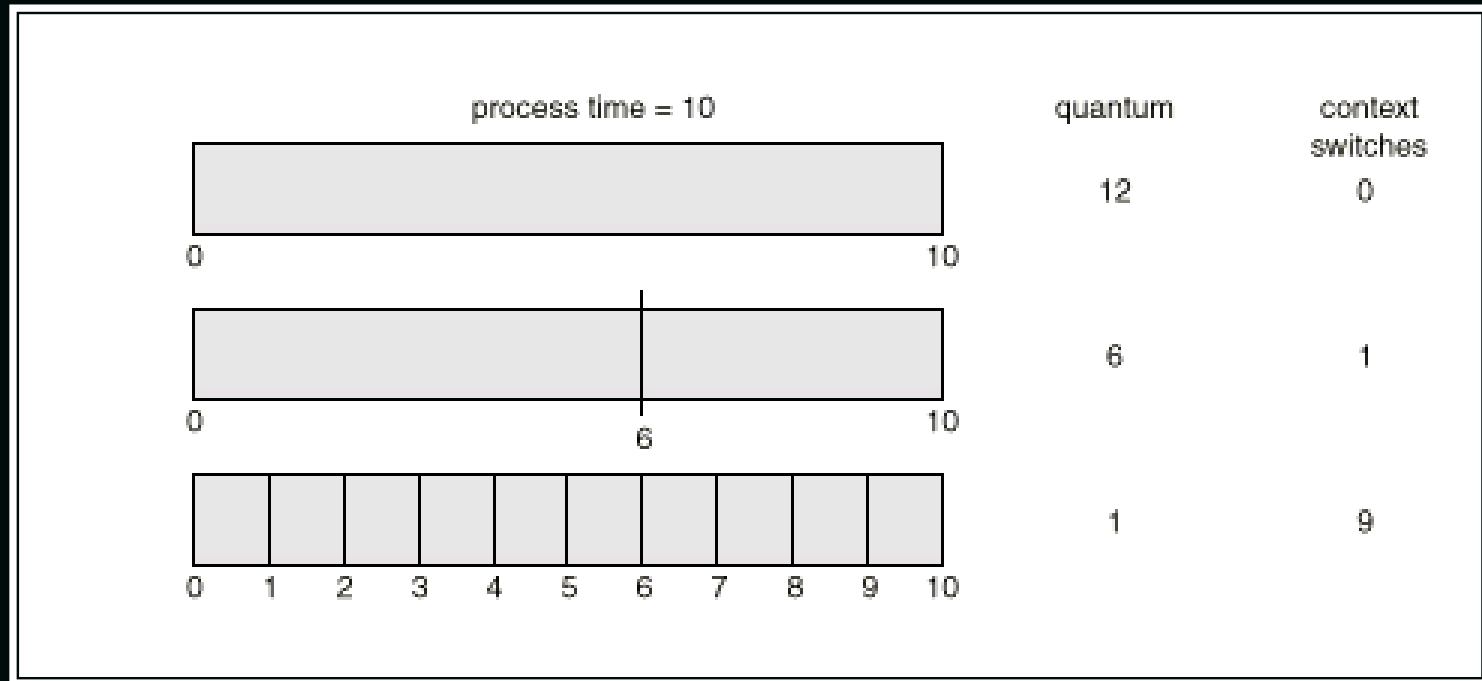


Timeslice : 1 ms



Timeslice : 20 ms

Time Quantum and Context Switch Time



Problem : Round Robin is fair, but uniformly inefficient.

Solution: Introduce priority based scheduling.

Priority based scheduling

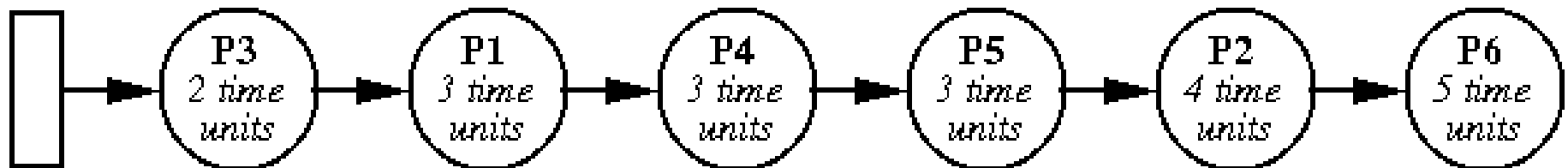
- Processes are allocated to the CPU on the basis of an externally assigned priority.
- Key to the performance of priority scheduling is in choosing priorities for the processes.

Problem: May cause low-priority processes to starve.

Solution: (AGING) Starvation can be compensated if priorities are internally computed. Priority assignment function - amount of time process has been waiting. Longer a process waits, higher its priority . This could eliminate starvation.

Shortest Job First

- Sorted according to job size
- Shortest job assigned highest priority





**IF YOU DON'T
BUILD YOUR DREAM,
SOMEONE ELSE WILL
HIRE YOU TO HELP THEM
BUILD THEIRS!**

-DHIRUBHAI AMBANI-

SRTN / Shortest time to completion

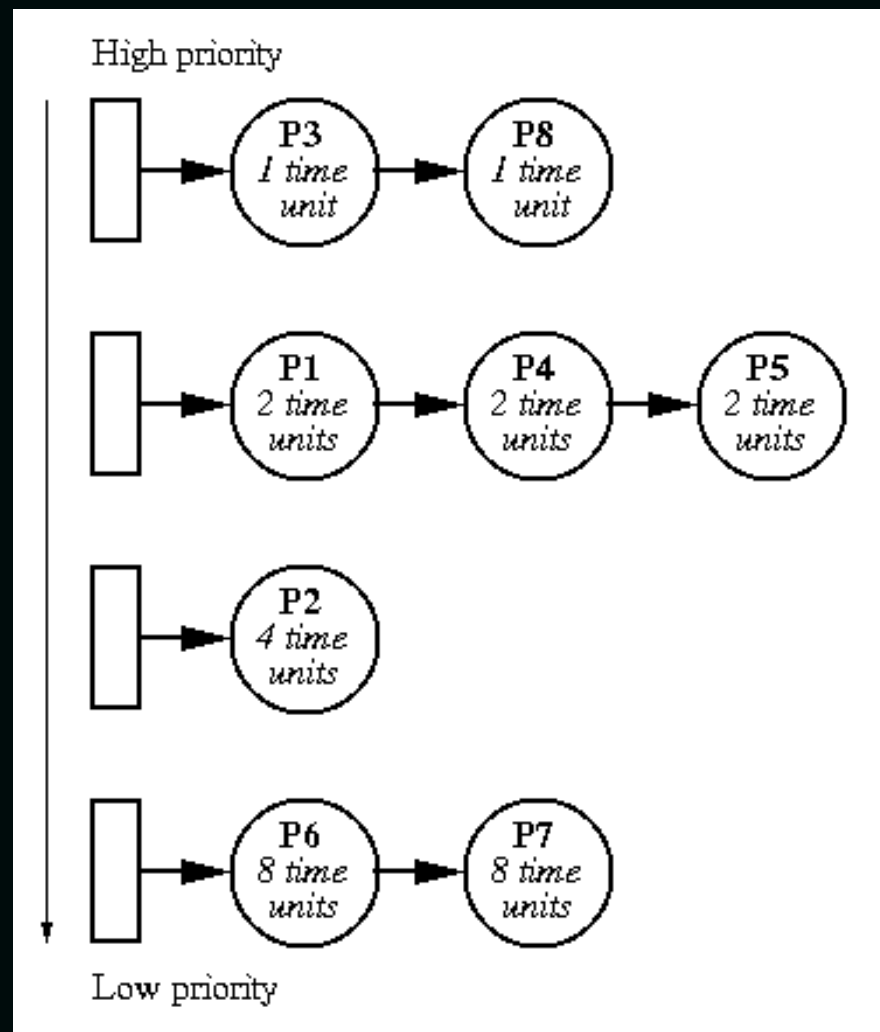
- Priorities are assigned in inverse order of time needed for completion of entire job
- Minimizes average turnaround time
- Optimal when all jobs arrive simultaneously
- Store estimated value in PCB for the current burst, and compare with actual value
- Knowledge of future needed

Problem : Starvation of large processes

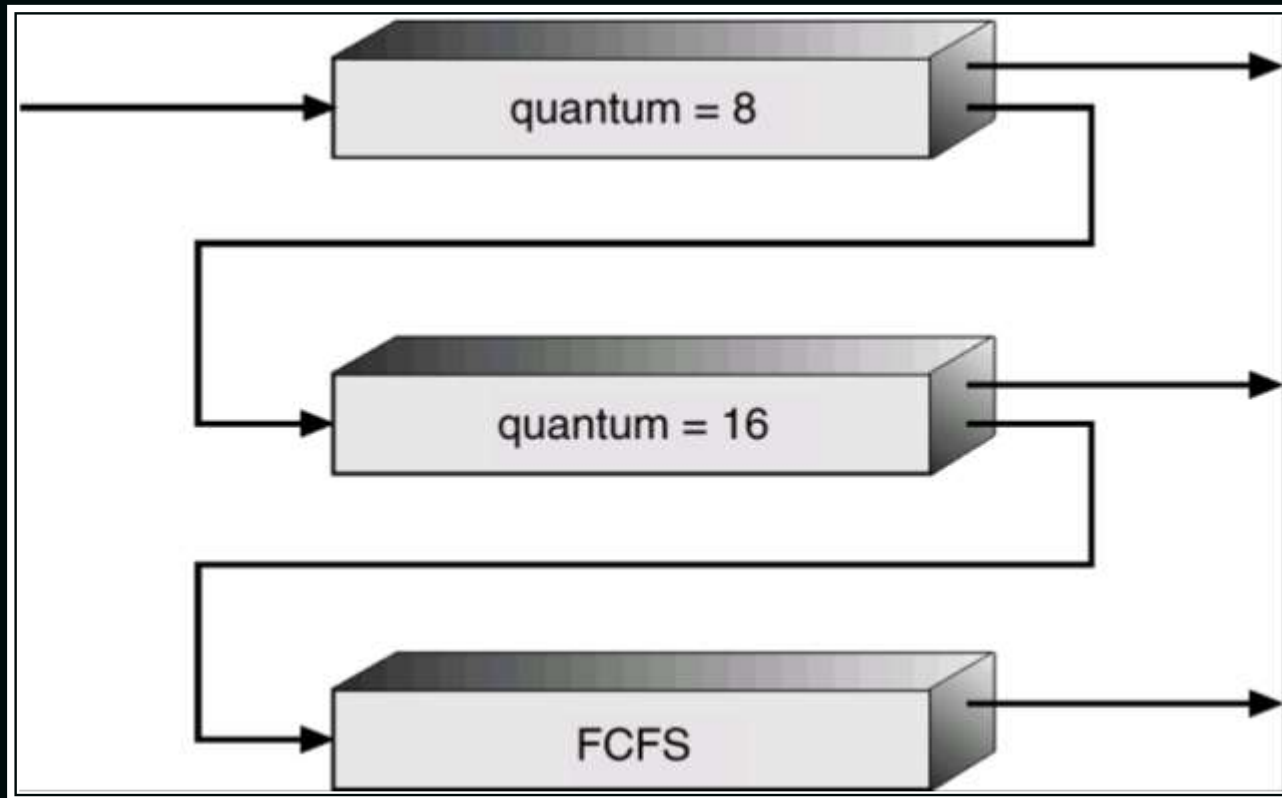
Solution :: Multi-Level Feedback Queues

Multilevel feedback Q

- Popular in interactive systems
- A Q for each priority level
- Efficiency and response time tackled
- New process added at the end of highest priority Q
 - It is allotted single time quantum when it reaches the front
 - If it yields CPU within the time quantum, it is moved to the rear
 - If not, it is placed at the rear of the next queue down
- Dispatcher selects from head of highest priority queue
 - A job that “succeeds” moves up
 - A job that “fails” moves down
- Eg: CTSS in MIT, Fairshare in Unix



Multilevel Feedback Q...



Multilevel Feedback Q...

- A process can move between the various queues (aging implemented this way).
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service

Problem :High Complexity

Solution : ??

5 jobs P1 through P5 arrive at a computer at almost the same time. Their running times and priorities are given below. Draw Gantt charts and determine the average process turnaround-time and the average waiting time for:

<u>Process</u>	<u>CPU Burst Time</u>	<u>Priority</u>	<u>A.T</u>
P1	10	3	0
P2	5	1 (lowest)	1
P3	4	5 (highest)	2
P4	9	2	3
P5	3	4	3

- Round robin scheduling with time slice = 2 time units.
- Shortest job first.
- Priority scheduling with preemption.
- Round robin scheduling with time slice = 2 time units and process switching overhead = 1 time unit.
- First come, first served scheduling.

Question

Assume that the following five processes arrive at time 0, 1, 2, 3 and 4 respectively, with the length of the CPU burst time given in milliseconds:

<u>Process</u>	<u>CPU Burst Time</u>
P1	6
P2	13
P3	7
P4	5
P5	15

Consider FCFS, SJF and RR (quantum=4) scheduling algorithms and determine the average turnaround time and average waiting time in each case, with the help of Gantt charts.

Review

- Types of processes
- Process relationship
- Process states
- Types of schedulers, difference between them
- Scheduling
- Criteria to evaluate scheduling algorithms
- Role of Time slice

Review

- Scheduling with / without
 - Preemption
 - Priority
- Scheduling algorithms

Process Scheduling
END